

GIT 简明教程





Table of contents



01

GIT是什么

What is git?

02

GIT的核心操作

Several key commands in git

03

稍作拓展

More useful commands

04

一些说明

Some explanations

01

GIT是什么

--世界上最先进的分布式版本控制系统

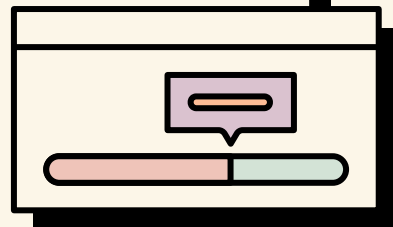


>>>> GIT

Git 是一种分布式版本控制系统，用于管理软件项目的源代码。它是由 Linux 之父 [Linus Torvalds](#) 开发的，并已经成为了现代软件开发领域中最流行的版本控制系统之一。

Ps: 分布式是指每台本地机器上都有一个完整的仓库副本，而不是由一个统一的中央服务器作为版本控制中心。

Meet us



举个栗子

想删除一个段落，又怕将来想恢复找不回来怎么办？有办法，先把当前文件“另存为.....”一个新的Word文件，再接着改，改到一定程度，再“另存为.....”一个新文件，这样一直改下去，最后你的Word文档变成了这样：

而当你再次想去寻找某个版本的文件时，往往已经忘记到底谁是谁了，看着一堆乱七八糟的文件，想保留最新的一个，然后把其他的删掉，又怕哪天会用上而不敢删，更重要的是如果此时要和同事共同处理这个项目，于是你把文件Copy到U盘里给她，然后，你继续修改Word文件。一天后，同事再把Word文件传给你，此时，你必须想想，发给她之后到你收到她的文件期间，你作了哪些改动，得把你的改动和她的部分合并，这不仅麻烦还容易出错。



此时你有这样一台高效运作的版本管理工具：



- Git就是这样一个工具，它Git 允许开发者在本地创建代码仓库，进行文件修改、提交等操作，直到准备好与远程仓库同步。
- Git 提供强大的分支管理和功能，可以在不干扰主分支的情况下开发新功能或修复 bug，且便于团队协作。
- Git 使用 SHA-1 哈希值确保版本数据的完整性，防止数据丢失

GIT 中的 核心操作

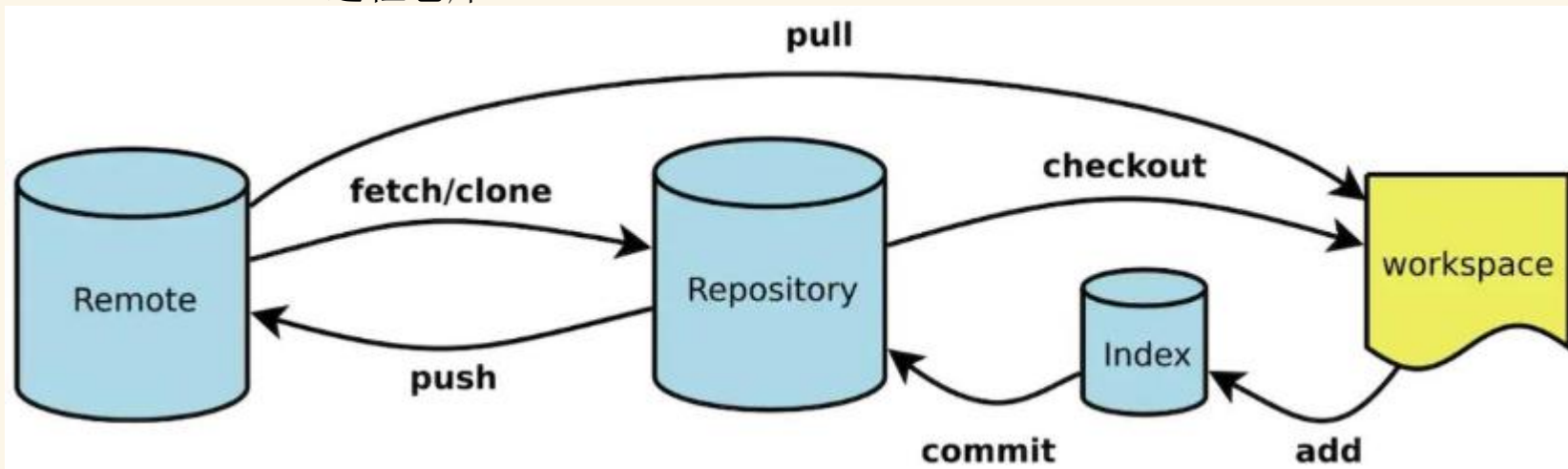
>>>>>

~~~~~  
.....



# 一张很经典的图：

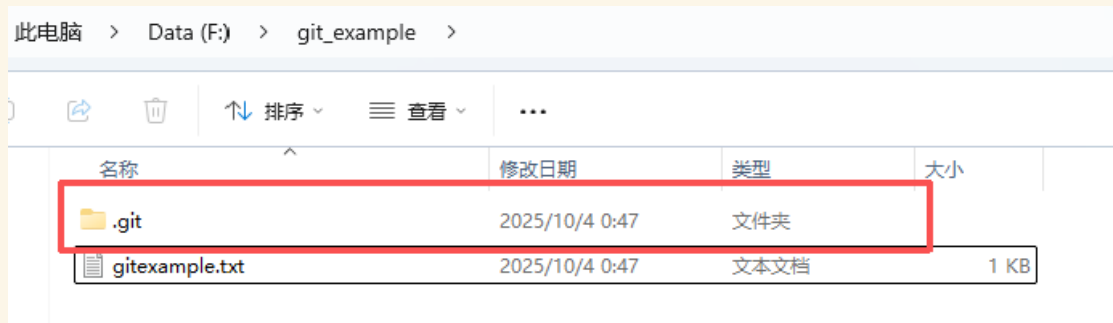
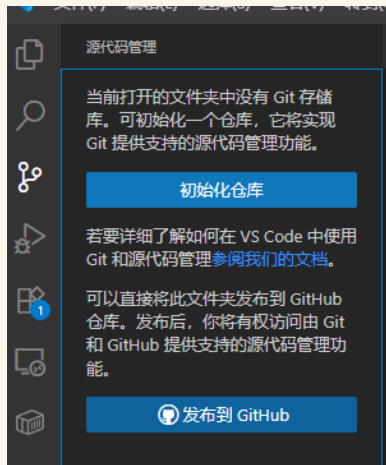
- Workspace:本地工作区
- Index:暂存区
- Repository: 本地仓库
- Remote: 远程仓库





# 初始化仓库：

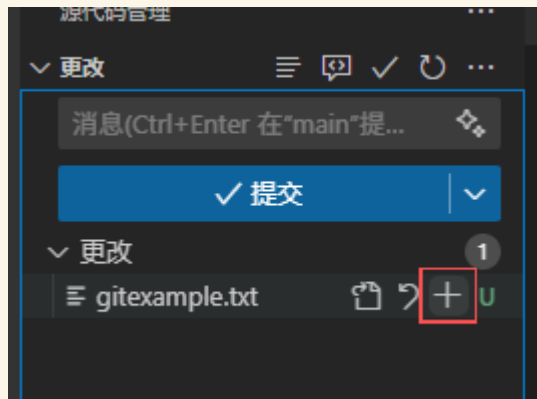
- git init



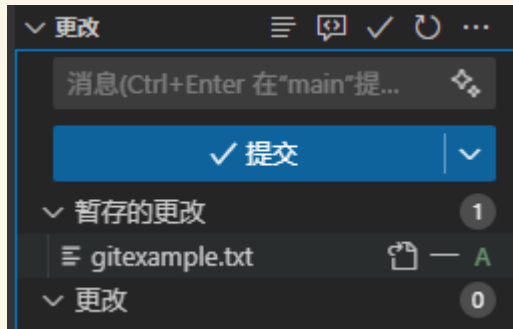
# 从工作区到暂存区：**add**

- `git add <file_name>` 或 `git add .`

```
(base) PS F:\git_example> git add gitexample.txt
```



- Vscode中未跟踪的文件会自动显示为绿色
- 点击这个小加号即可将文件放入暂存区

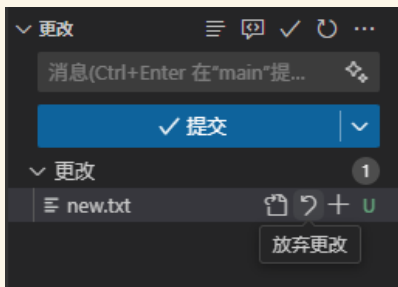


# 撤销工作区的更改

- 当你想尝试某种新的代码思路但是写完发现效果并不好想放弃这次的更改:
- `git checkout .` 或者 `git restore .` 可以帮你直接撤销工作区内所有文件从上一次`add`至今的修改（即未暂存的修改）

这里的`.`表示所有，你也可以指定某个文件 如`git checkout its.txt`，即撤销`its.txt`的更改

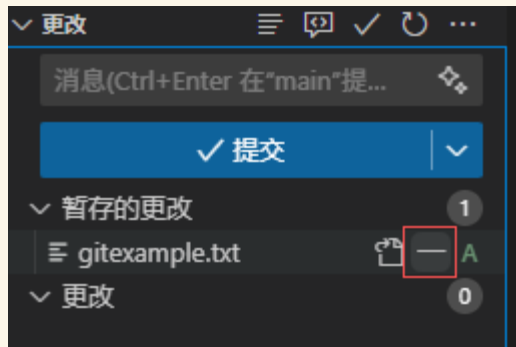
但是注意这个操作不会删除你在上一次`add`之后新建/新出现的文件，如果你不想要这个文件了可以手动删除或者先`commit`再用`reset --hard`删除



嗯 其实使用vscode的放弃更改是非常方便的操作

# Git add之后想反悔怎么办

- `git reset` 即可将已经添加进暂存区的文件吐出来（吐回工作区）
- `git reset --hard`则不止是将文件吐出暂存区，还会将其在工作区的痕迹一起嘎掉

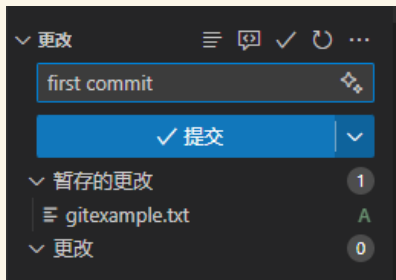


还有：`git rm --cached <file_name>`

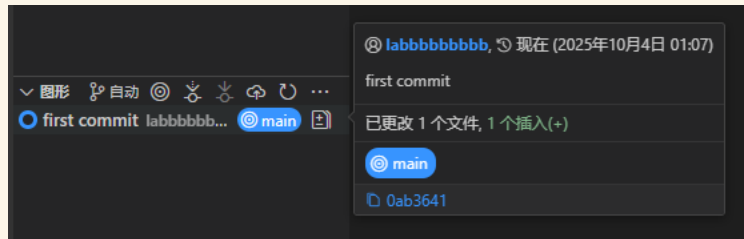
这个操作会将指定的文件整个移出暂存区（而不止是上次提交后刚做的更改）

# 从暂存区到本地仓库: **commit**

- `git commit -m` "add some message to mark this commit"



- 这里的提交信息必须填 用来对这次提交作必要的解释
- 上面那句默认提交所有更改, 也可以只提交指定文件的修改, **commit**加指定文件名即可
- 一旦提交, 你的仓库就会生成一个提交节点(可用**git status** 与 **git log**查看)
- `git log --graph --pretty=oneline --abbrev-commit`

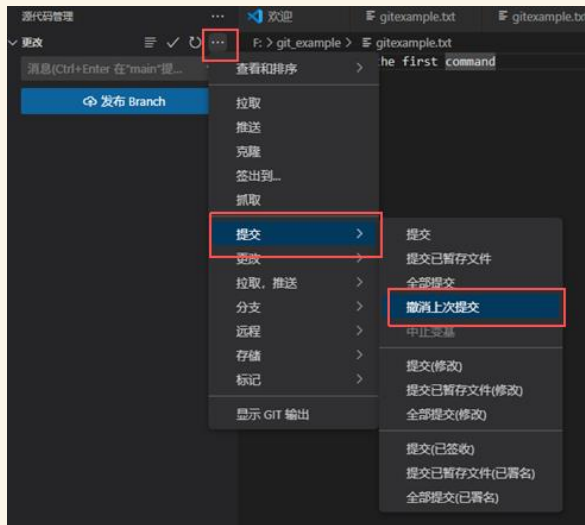


```
(base) PS F:\git_example> git status
On branch main
nothing to commit, working tree clean
(base) PS F:\git_example> git log
commit 0ab364122c3f35955ace3740a0f12e77dd1ccfb5 (HEAD -> main)
Author: labbbbbbbb <724757025@qq.com>
Date: Sat Oct 4 01:07:11 2025 +0800

    first commit
```

# 想要撤销某次提交怎么办：

- `git reset --soft HEAD~1` 即可将本次commit丢回暂存区（相当于下图vscode的操作）



- `git revert <hash_of_commit>` 是一种不同的回退方法，它通过删除本次的commit的所有更改并且创建一个新的提交来回到前一个状态，好处是它不仅回到了之前的状态，同时也保存下了被“删除”的那一次commit

# 从本地仓库到远程仓库：

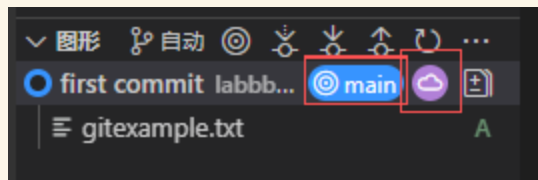
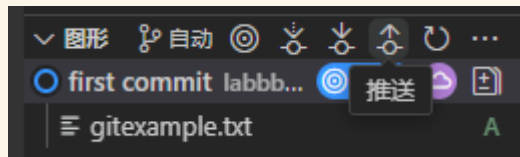
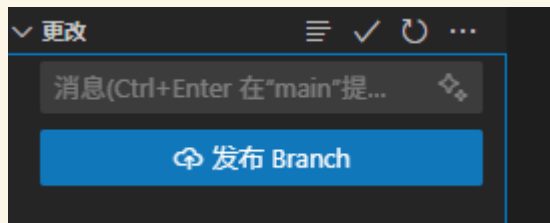
- push 包括后面的pull 都需要关联远程仓库，vscode中关联远程的操作见群文件中git的配置
- 也可以使用命令行：
- `git remote add <name_of_repo> <url_of_repo>`

<name\_of\_repo>是自己给仓库的命名，默认一般叫origin  
<url\_of\_repo>就是在gitee或github上面复制下来的那个网址

- `git remote -v` 查看远程仓库

# 从本地仓库到远程仓库: **push**

- `git push origin main` 其中origin是远程仓库的名字 main是本地分支的名字

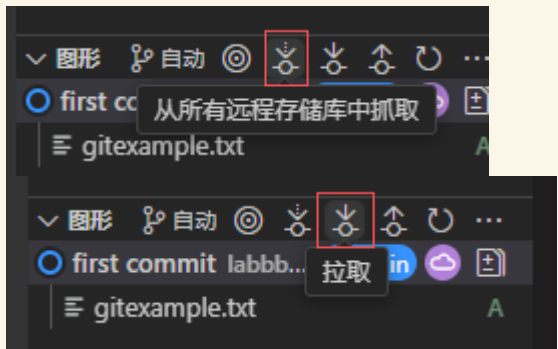


- 推送完成后可以看到这个紫色的云标志，这表示你的这个提交已经上传到远程
- 蓝色的main标志表示你的main分支的HEAD指针位于这次提交上
- 另：使用`git push origin main -f` 去强制将远程仓库变为和当前的本地仓库一模一样（包括所有的提交关系、分支关系）
- 注意上传到远程时有文件大小限制



# 从远程到本地仓库: **pull**

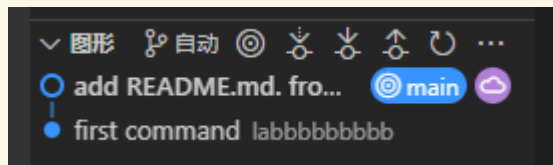
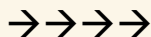
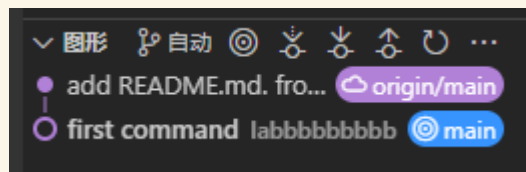
- 如果队友在远程修改了仓库的内容, 而我需要在ta修改后的基础上继续编辑, 那么我需要将远程仓库的内容先拉取下来
- pull = fetch+merge
- **git pull origin main**      origin:远程仓库名    main:远程分支名



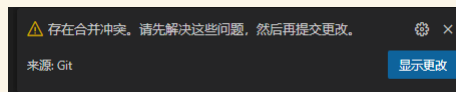
- 上面那个图的操作就是**fetch**, 可以可视化远程与本地的差别 (让本地仓库感知远程的变化)
- 注意**merge**的过程是可能会产生冲突的, 这是需要手动解决冲突部分才能继续merge
- [git fetch & pull详解 - Runner Jack - 博客园](#)

# 示例：从远程拉取回**readme**文档

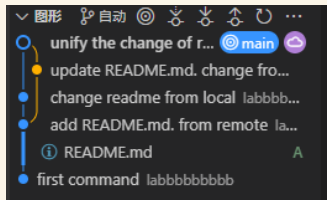
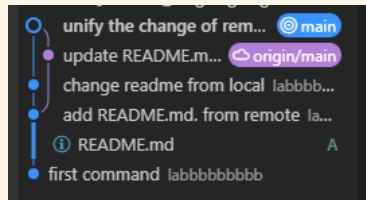
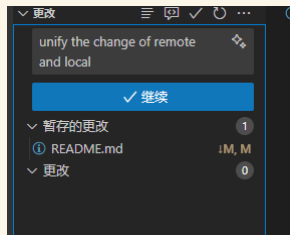
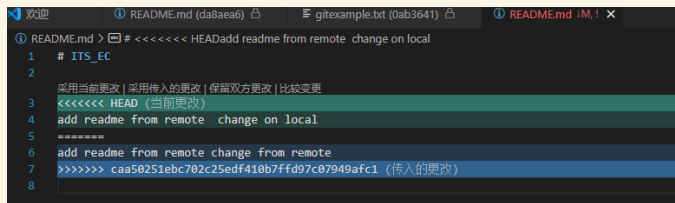
- 点击“从所有远程库中抓取”，看这个图标，紫色代表远程分支，蓝色代表本地分支，远程发生变动之后其新的提交领先于本地分支，需要用pull拉回来



- 若同时对本地和远程的同一个文件进行不同的修改，合并就会产生冲突：此时直接修改冲突文件，解决冲突，再重新将这个文件提交即可  
(作为merge节点的一次commit)



下图分别为 对冲突部分进行统一、重新提交解决冲突后的文件(图2和3)、再次将merge的提交上传到远程



# pull遇到fatal: refusing to merge unrelated histories?

- 原因是此时你的本地仓库和远程仓库分支关系不同，具有不同的提交历史
- 解决方法：
- (1)允许不相关历史提交并强制合并，这时你会发现该拉取的确实都拉取下来了，但你的commit支路就断成了两截，因为pull下来的commit历史与本地的无法融合

```
git pull origin master --allow-unrelated-histories
```

- (2)放弃远程已有的历史，一切按照本地的来：

```
git push origin main -f
```

意思是先把本地仓库强制推送至远程，这会让远程变得和本地一模一样。

# 将远程某个仓库复制到本地: **clone**

- `git clone <repo_url>` url是远程git仓库地址, 如<https://gitee.com/xxxx/xxx.git>,即之前添加远程仓库时用的那个url
- Clone会把整个远程仓库copy到本地仓库
- Clone不需要本地关联这个远程仓库, 只要有网址就能直接克隆



21°



# 稍作拓展



Reset?分支? Head detached?rebase?



# 版本回退: **reset**

- 正如之前提到的，回退可以使用**reset**，但**reset**也分几种效果

- `git reset --hard HEAD~n`

`--hard`表示回退的同时删掉所有工作区中的更改（谨慎！删了就再也找不回来了！）  
`HEAD~n`中`n`是整数，代表回退的提交数目，`n=1`时为撤销上一次提交，以此类推

- `git reset --soft HEAD~n`

`--soft`表示回退之后把**commit**的东西吐回暂存区

- `git reset HEAD~n`

表示回退之后把**commit**的东西吐回工作区（未暂存）

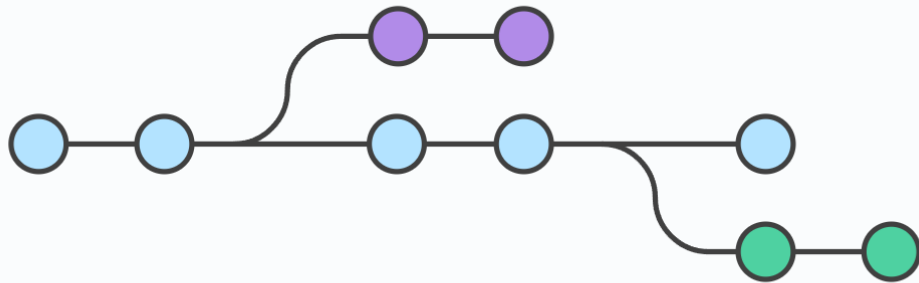
其中`HEAD~n`也可以直接改为某次提交的哈希值，代表回到那个提交后的状态。

# GIT中的分支管理：

Git 分支管理是 Git 强大功能之一，能够让多个开发人员并行工作，开发新功能、修复 bug 或进行实验，而不会影响主代码库。

几乎每一种版本控制系统都以某种形式支持分支，一个分支代表一条独立的开发线。

使用分支意味着你可以从开发主线上分离开来，然后在不影响主线的工作时继续工作。



Git 分支实际上是指向更改快照的指针。

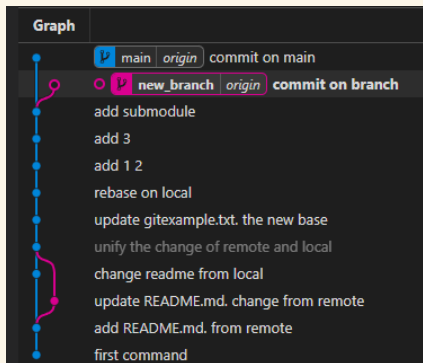
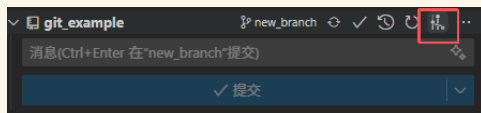
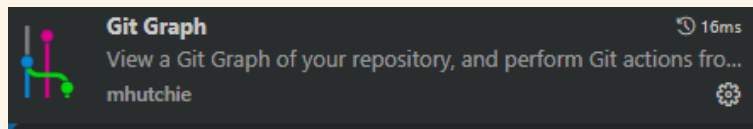
有人把 Git 的分支模型称为**必杀技特性**，而正是因为它，将 **Git** 从版本控制系统家族里区分出来。

- 对于分支的指针的理解看这个：[创建与合并分支 - Git教程 - 廖雪峰的官方网站](#)
- 对于两个存在冲突的分支的合并看这个：[解决冲突 - Git教程 - 廖雪峰的官方网站](#)
- 更多关于git branch和git checkout的命令行：[Git 分支管理 | 菜鸟教程](#) 也是上图来源
- 深入理解checkout：[git深入理解\(六\)：git checkout详解-CSDN博客](#)

# GIT中的分支管理:

- 推荐一个非常好用的git插件: git graph

[GitGraph 插件使用 - 知乎](#)



支持用图形化管理分支, 双击某分支即可切换(**checkout**)到该分支上

右键可以直接删除/重命名/向远程推送该分支/新建另一个分支

可以看到所有本地分支、远端分支、本地|远端分支的关系

等等等等



# 如何回到某个提交节点并新建分支？ 为什么会出现**head detached**？

- `git checkout`的作用：一是切换分支，二是撤销修改

其名为“签出”，本质上是将当前的**HEAD**指针指向所切换的目标分支的最近一次**commit**，从这个**commit**恢复暂存区和工作区，从而我们眼前的工作区变成了**checkout**所签出的位置。

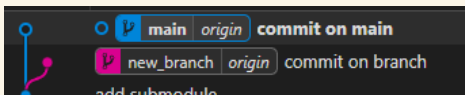
通过**`git checkout <commit-hash>`** 或者**`git checkout HEAD~n`** 你可以切换到特定的提交状态，此时**Git HEAD** 不再指向某分支，而是指向某次具体的提交，这将使你进入**head detached**状态，只能查看历史记录，而不能进行分支操作。通常情况下，不建议在分离头指针状态下工作，因为更改可能会丢失。

- 如果你就是想回到这个节点并再此基础上再做开发，使用**`git checkout -b <new_branch_name>`**即可在当前的提交节点上新建分支并在这个节点上继续工作（可以让我们离开头指针分离状态，因为头指针指向了新的分支头）
- 如果你只是不小心使用了这个命令进入了头指针分离状态并且不打算新建分支：
  - 使用 **`git switch -`** 去撤销这个操作,让被分离的**HEAD**回到原来的位置。
  - 使用 **`git checkout <branch_name>`**直接切换分支即可

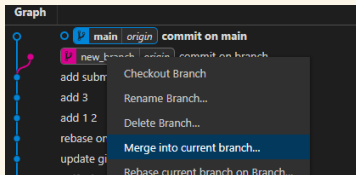
详见：[【Git】：遇到 Detached HEAD 怎么办？-腾讯云开发者社区-腾讯云](#)

# 两个分支的融合: **merge**

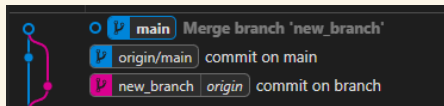
- 由上所述, 我们会发现很多时候我们会有新建多分支的需要, 但有的分支处理完之后是可以合并的, 这时候我们可以使用**merge**来进行融合。
- `git merge <branch name>` (将<branch name> 分支与你当前所在的分支融合)
- 也可以使用`git graph`操作:



如图, 当前位于main分支



右键new\_branch分支, 选择merge...  
解决可能遇到的冲突, 可以得到:



Pull的本质是把远程分支抓取下来并融合FETCH指针与当前分支  
并且merge的特点就是会产生多分支的分岔交融点, 好处是它保留了所有的提交信息, 坏处是会让提交记录看起来比较混乱

# 另一种融合分支的方式：变基操作

- 变基的概念：[git rebase 详解（图解+最简单示例，一次就懂）-CSDN博客](#)



its\_ec / gitexample.txt

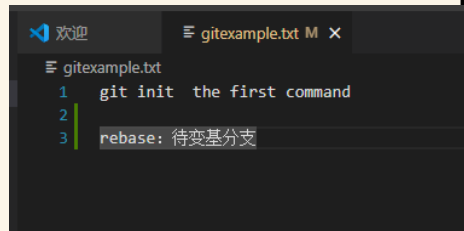
gitexample.txt 45 Bytes

labbbbbbbbbb 提交于 1分钟前 · update gitexample.txt the new base

```
1 git init the first command
2
3 rebase:基分支
```

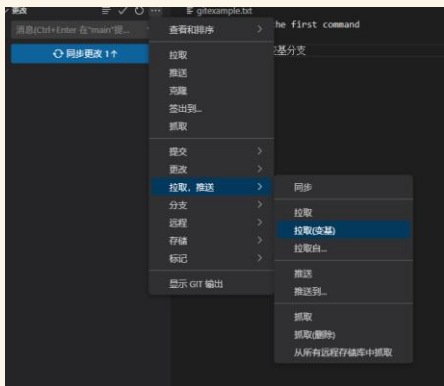
- 例如：提交远程的修改：

提交本地的修改：



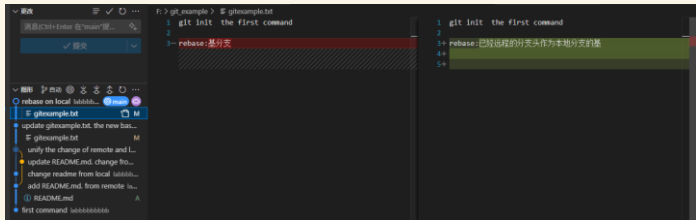
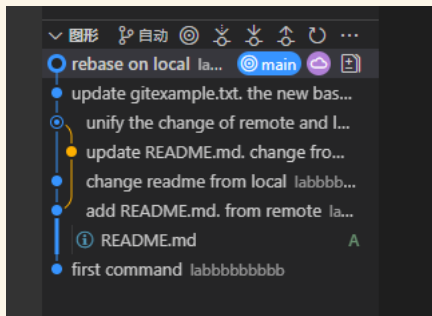
```
gitexample.txt M x
gitexample.txt
1 git init the first command
2
3 rebase: 待变基分支
```

- 使用变基方式拉取远程：git pull origin main --rebase



# 让分支美观一点：变基操作

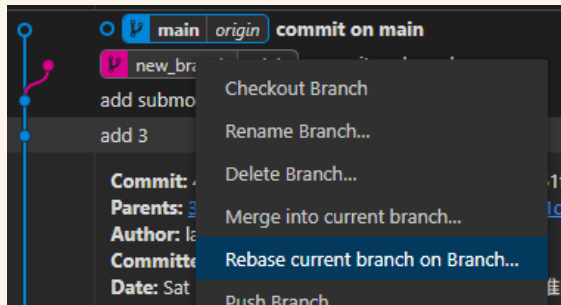
- **Rebase**的效果如下（最新的两条提交记录）



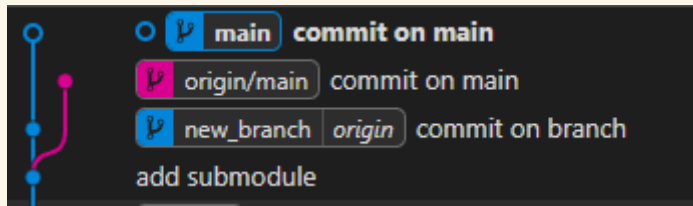
- **Rebase on local**是原来在本地的提交
- **Update**（第二条）.....是在远程的提交
- 可以看见现在远程的提交已经作为本地提交的前置提交状态，最后得到的是**commit**记录是一条直线而不是像前面一样的分岔和交叉。
- 最新的提交记录的更改显示的是从左边红色部分变为右边绿色部分，而左边红色部分恰是我们先前在远程所作的更改，由此可以理解变基的逻辑：将远程已有的提交作为新的基，本地提交将叠加在这个基上面，由此既融合了双方的更改又是**git log**只留下一条直线

# 让分支美观一点：变基操作

- 在merge那里我们讲了本地两个分支的融合，也提到merge会产生分岔和交融，同样的栗子我们也可以使用变基

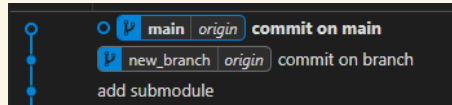


在git graph中，右键new\_branch分支，选择将current branch(这里为main)变基到new\_branch上，可以得到：



看蓝色线条，可知本地记录已经变为一条直线（红色为远端分支，说明远端的状态与本地的main分支有不同）

若要将此时的状态推送到远端origin/main,使用git push origin main -f



# 让分支美观一点：变基操作

- 变基也可以用本地分支与分支之间的合并，用来简化分支关系
- 注意：在pull的时候可以用rebase，但是往公共分支上合并代码的时候不要用rebase，而要用merge

1. **拉公共分支最新代码——rebase**，也就是git pull -r或git pull --rebase。这样的好处很明显，提交记录会比较简洁。但有个缺点就是rebase以后我就不知道我的当前分支最早是从哪个分支拉出来的了，因为基底变了嘛，所以看个人需求了。**总体来说，即使是单机也不建议使用。**
2. **往公共分支上合代码——merge**。如果使用rebase，那么其他开发人员想看主分支的历史，就不是原来的历史了，历史已经被你篡改了。举个例子解释下，比如张三和李四从共同的节点拉出来开发，张三先开发完提交了两次然后merge上去了，李四后来开发完如果rebase上去（注意，李四需要切换到自己本地的主分支，假设先pull了张三的最新改动下来，然后执行<git rebase 李四的开发分支>，然后再git push到远端），则李四的新提交变成了张三的新提交的新基底，本来李四的提交是最新的，结果最新的提交显示反而是张三的，就乱套了，以后有问题就不好追溯了。
3. **正因如此，大部分公司其实会禁用rebase，不管是拉代码还是push代码统一都使用merge，虽然会多出无意义的一条提交记录“Merge ... to ...”，但至少能清楚地知道主线上谁合了的代码以及他们合代码的时间先后顺序**

# Git仓库中的git仓库：子仓库

- 如果你的仓库中的某个子文件夹又含有一个.git仓库，此时要直接add是会报错的

```
warning: adding embedded git repository: yolov5
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> yolov5
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached yolov5
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config advice.addEmbeddedRepo false"
```

`git submodule add <repo-url> [<path>]`

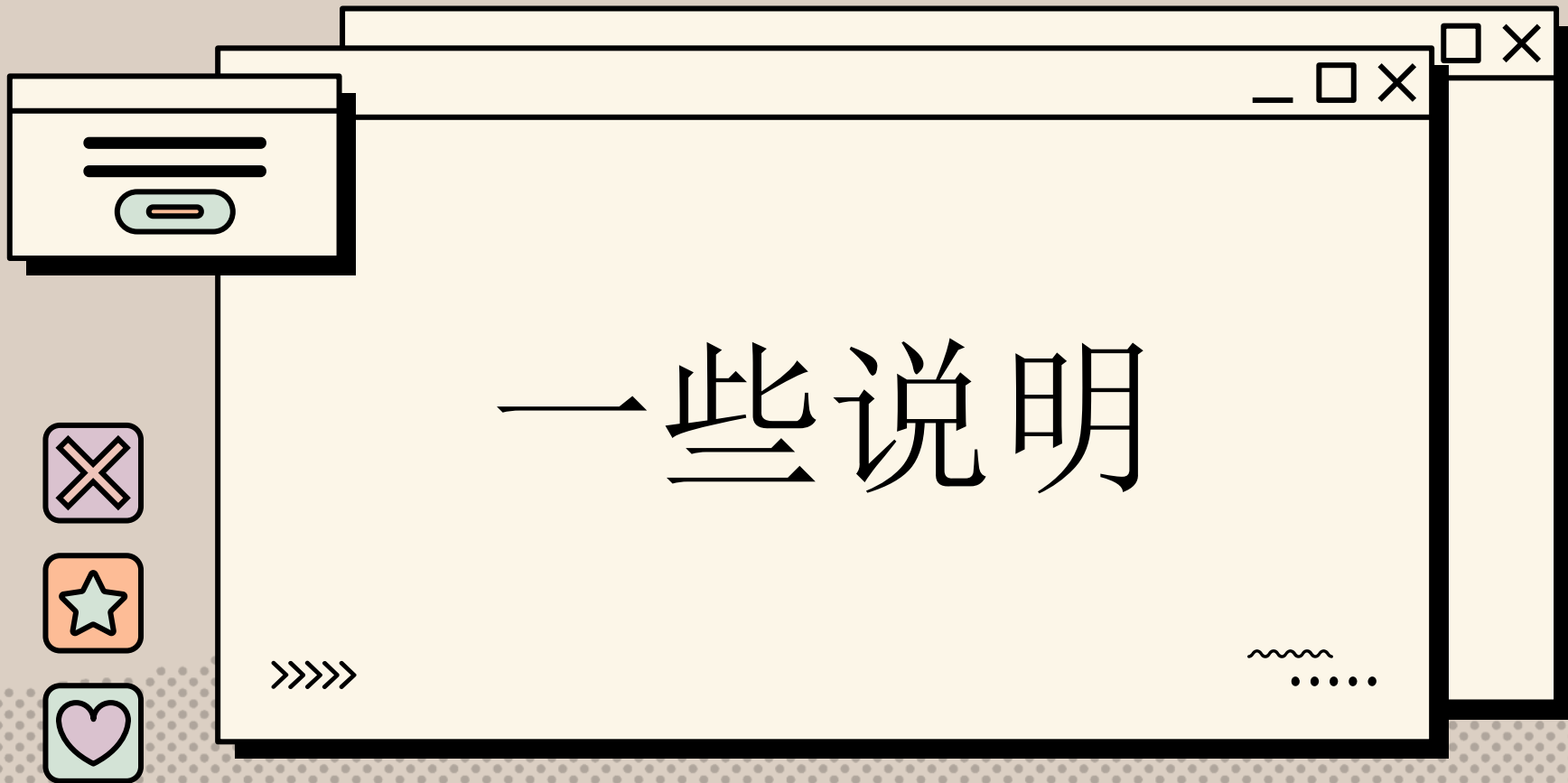
repo-url是远程仓库地址，path是本地文件路径

[一文带你彻底搞懂 git submodule 原理和用法 - 知乎](#)

# Git提交的标记: **tag**

- [Git基础 - git tag 一文真正的搞懂git标签的使用-CSDN博客](#)
- 在一个项目中为了保证记录每一个功能的修改你会进行很多次提交，但其中可能只有某几次提交是具有划版本的重要意义的，这时你可以使用 `git tag _标签名_ _提交版本号_` 来为某一次提交做标签，如 `git tag v1.0 c23ad3` 等等。
- 推送标签时使用 `git push origin --tags`







# 食用指南

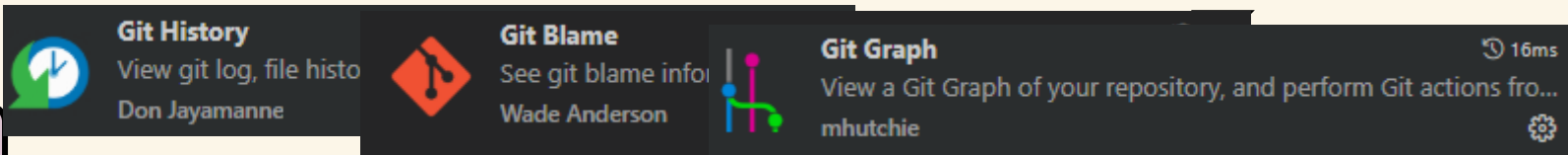
>>>>

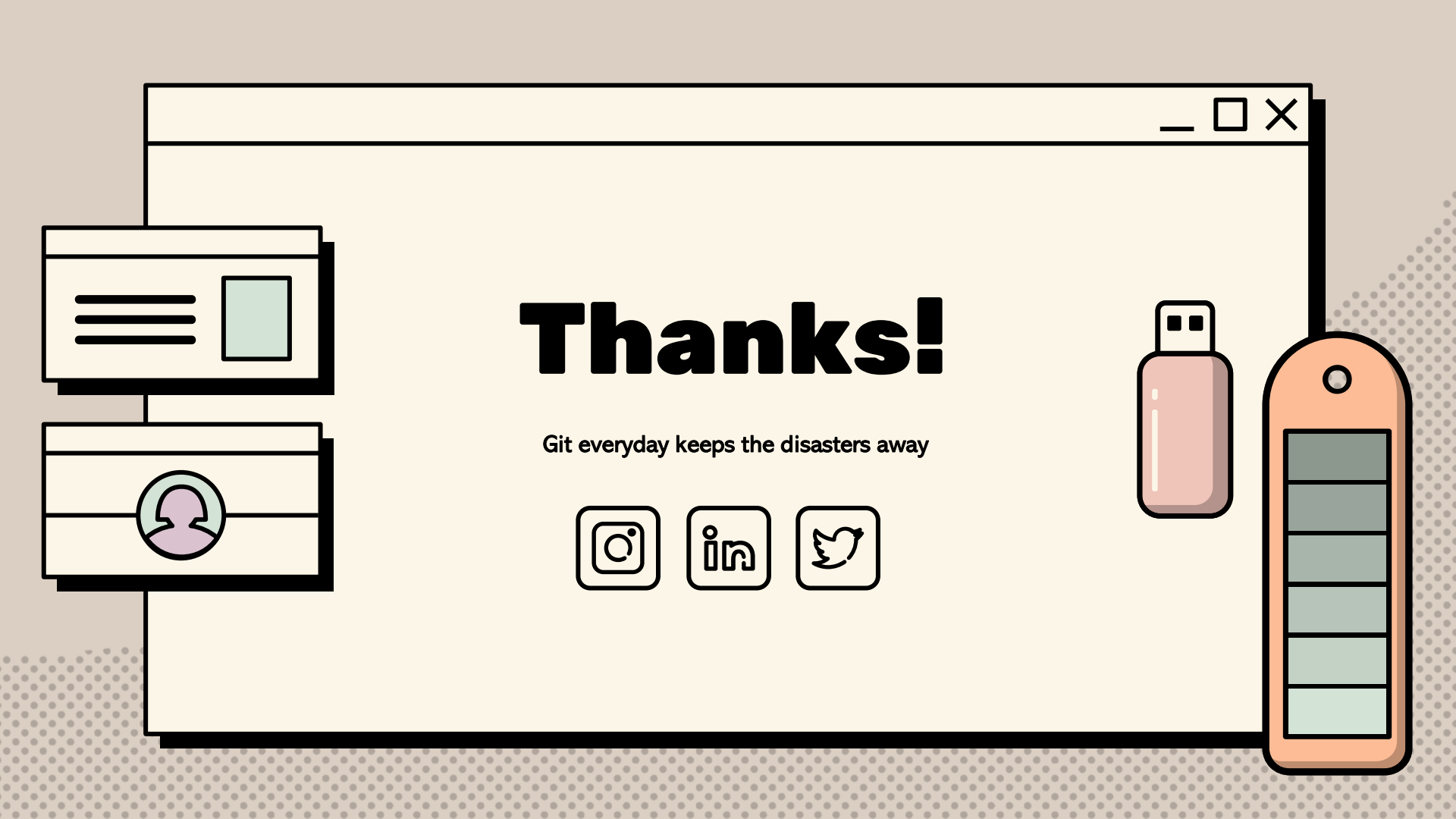
1.上面所列出的命令远不是git的全部内容，鼓励大家继续去探索

2.但是如果只是日常使用的话 核心命令部分基本够用 剩下的可以等需要用了再去学

3.上面所涉及的命令基本都包括了vscode中的操作和命令行两种方式，大家可以选用自己喜欢的方式，vscode是可视化的，好上手，但是对有些操作来说命令行可以更灵活。

4.最后再推荐一下这几个好用的插件,具体用法可以上网细查





# Thanks!

Git everyday keeps the disasters away

